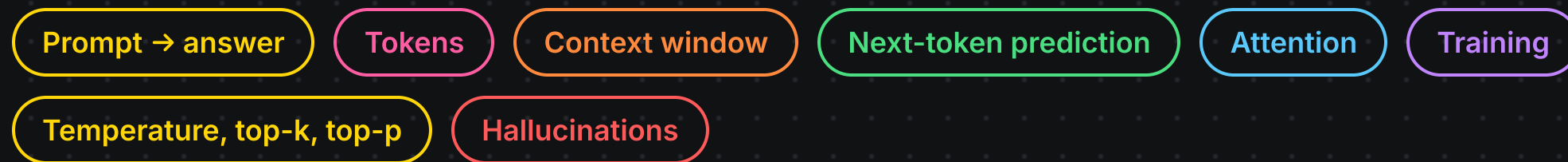


Day 02

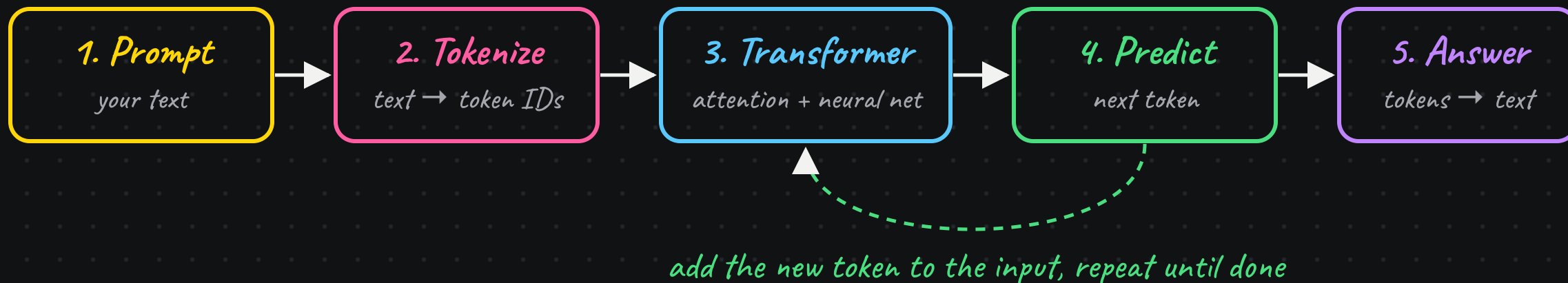
How LLMs actually work

What happens between sending a prompt and getting an answer, why the output changes from run to run, and why models sometimes make things up.



the big picture

From prompt to answer



The capital of India is → New → Delhi

The model does not fetch a ready-made sentence. It picks **one token at a time**, each based on everything before it.

what the model actually reads

Tokens and tokenization

A **token** is a chunk of text: a word, part of a word, a space + word, or punctuation. The model only sees numbers, so each token maps to an **ID**.

I love machine learning !

[40, 3047, 7342, 7524, 0]

Hello world

[13225, 2375]

1 token $\neq$ 1 word. Notice the space is part of the token: " love", not "love".

why care? You pay per token, and limits are in tokens.

Same idea, different count

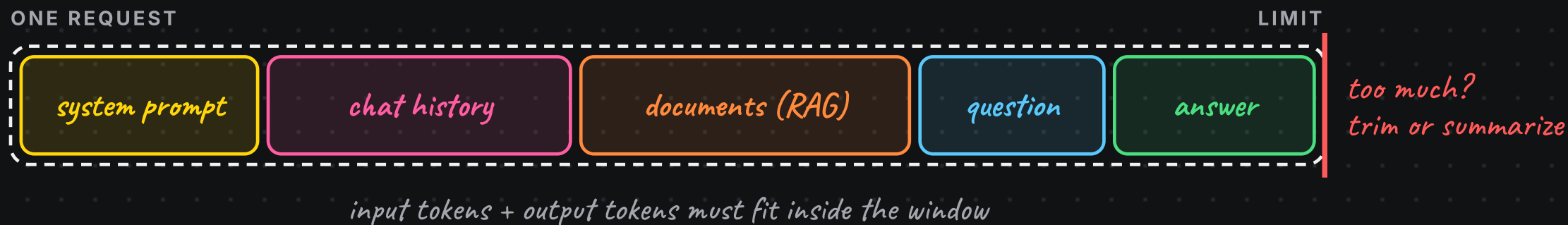
TEXT	TOKENS
I love machine learning!	5
मुझे मशीन लर्निंग बहुत पसंद है!	10
Mujhe machine learning bahut pasand hai!	11
def add(a, b): return a + b	11
Pneumonoultramicroscopic...	15

Real counts from `count_tokens.py` (tiktoken, o200k_base).
Other tokenizers give other numbers.

the model's working desk

Context window

The maximum number of tokens the model can handle in **one request**. Everything you send, plus the answer it writes, has to fit.



Small desk

Less fits at once. Long chats or big PDFs must be cut down.

Big desk

More history and documents fit, but every token still costs money and time.

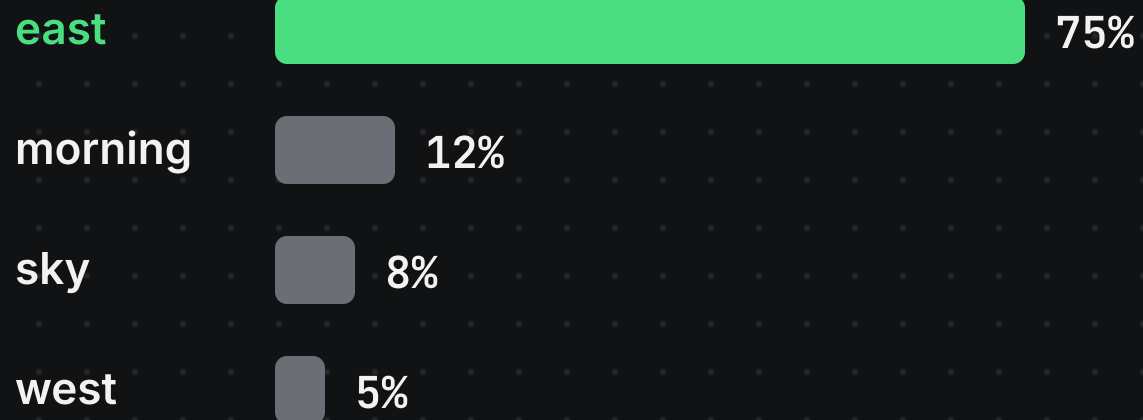
Not memory

A new chat starts with an empty desk. Memory across chats is something **we** build.

one token at a time

Next-token prediction

Prompt: The sun rises in the ___



Illustrative numbers, not real model output.

1. Score every token

The model gives a probability to every token in its vocabulary (hundreds of thousands).

2. Pick one

A **sampling** rule chooses the token. Temperature, top-k and top-p change that rule.

3. Repeat

Add it to the text and predict again, until a stop token or the max token limit.

how it uses context

Attention

For each token, attention decides **which other tokens matter** and mixes in their meaning, even when they are far apart.



Query

What am I looking for?

Key

What does each token offer?

Value

What does it pass on if picked?

how a model is made

How LLMs are trained

stage 1

Pretraining

Predict the next token across a huge amount of text. The model learns language, facts and patterns.

reading a giant library

stage 2

SFT

Supervised fine-tuning

Train on prompt → good answer pairs, so it follows instructions instead of just continuing text.

learning how to answer an exam question

stage 3

RLHF / DPO

Preference tuning

RLHF: humans rank answers, a reward model learns that, RL optimizes for it.
DPO: learn directly from good vs bad answer pairs.

learning which answers people prefer

stage 4

Reasoning training

RL that rewards correct multi-step solutions. At answer time, the model can spend more compute "thinking" (**test-time compute**).

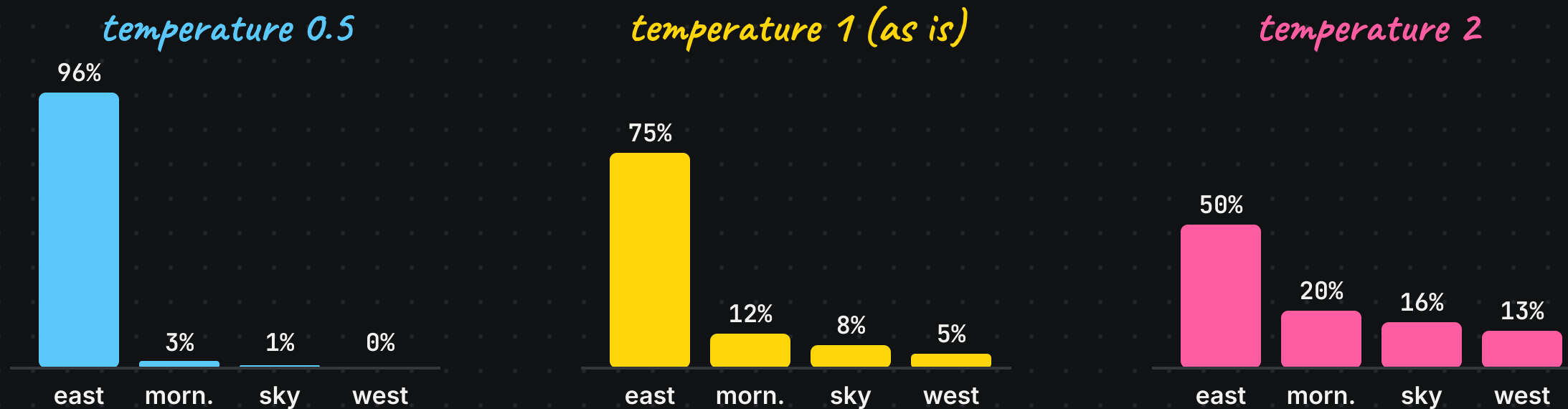
taking time to check your work

Not every model follows exactly this pipeline. More thinking helps on hard tasks, but it does not guarantee a correct answer.

controlling the pick, part 1

Temperature

Reshapes the probabilities **before** a token is picked. Low = sharper (safe, repeatable). High = flatter (varied, creative, riskier).



almost always "east" ← same model, same prompt → "west" now has a real chance

Illustrative numbers. Temperature 0 = always take the top token (close to repeatable, not a hard guarantee).

controlling the pick, part 2

Top-k, top-p and max tokens

TOKEN	PROB.	RUNNING TOTAL	TOP-K = 2	TOP-P = 0.9
east	75%	75%	✓ in	✓ in
morning	12%	87%	✓ in	✓ in
sky	8%	95%	✗ out	✓ in (crosses 90%)
west	5%	100%	✗ out	✗ out

Same illustrative numbers as before.

Heads-up

Reasoning models often lock these. OpenAI's gpt-6-luna only accepts temperature / top-p with reasoning set to none. OpenAI has no top-k (Claude and Gemini do).

SETTING	CONTROLS	THINK OF IT AS
temperature	How random the pick is	How predictable the answer is
top-k	Keep only the k most likely tokens	A fixed-size shortlist
top-p	Keep the top tokens until their total reaches p	A shortlist that grows or shrinks with confidence
max tokens	Longest answer allowed	Page limit. Too low cuts the answer off

Rule of thumb: change one knob at a time. Use low temperature for extraction, SQL and code; higher for brainstorming and copy.

try it yourself

Live demos

1 · Count tokens

English vs Hindi vs code: compare counts and where the splits fall.

```
# built-in examples
python count_tokens.py

# your own text
python count_tokens.py "Namaste duniya"
```

Also try in the browser: tiktokenizer.vercel.app

2 · Temperature 0 vs 1

Same prompt, 3 runs each. Watch which answers repeat.

```
# "3 creative names for an AI startup"
python temperature_demo.py

# add top-p, more runs
python temperature_demo.py --top-p 0.1 --runs 5
```

Needs OPENAI_API_KEY in day-02/code/.env

ask: Why do two sentences with the same meaning use a different number of tokens?

confident, fluent, and wrong

Why LLMs hallucinate

A **hallucination** is false or made-up information stated as fact. Ask for a paper that doesn't exist and you may get a real-looking title, authors and year.

Why it happens

- **Prediction $\neq$ verification.** It writes what is likely, not what is checked.
- **Missing context.** The fact isn't in the prompt or in training.
- **Vague question.** It assumes instead of asking.
- **Bad data.** Training text or retrieved docs can be wrong.
- **Fluent $\neq$ correct.** It sounds sure either way.

How we reduce it

- **RAG:** give it the right documents.
- **Citations:** ask for sources, then check them.
- **Tools:** look up live facts via APIs.
- **Permission to say "I don't know".**
- **Evals:** test against known answers.

remember: RAG **reduces** hallucinations, it doesn't remove them. Wrong doc retrieved, right doc misread, or a claim made up anyway.